


**UNIVERSITÉ DE POITIERS**

Master 2 – EUR Smart IoT

---

**RAPPORT DE TRAVAUX PRATIQUES**

Module : Machine Learning

# Système expert :

# Application à la résolution des Tours de Hanoï

---

**Réalisé par :**

– CHAIB Abdelfatah
– HAJJAJ Houda

**Encadrant :**

Pr. YOUNES Mohamad

Année universitaire 2026–2027

# Résumé

Ce travail pratique porte sur la conception et l'implémentation d'un système expert destiné à résoudre le problème des Tours de Hanoï. L'objectif est d'étudier le fonctionnement d'un système à base de règles, en mettant en œuvre une représentation des états, une base experte et un moteur d'inférence utilisant le chaînage avant.

Le système est développé en Python à l'aide de la bibliothèque NumPy. Le problème est modélisé à partir de trois pics et de trois palets de tailles différentes. Une première stratégie fondée sur un ensemble de règles à priorité fixe permet d'atteindre la configuration finale en 26 déplacements. Une seconde approche enrichit la base experte et adapte la sélection des règles afin de guider la résolution. Dans l'expérimentation réalisée avec trois palets, cette stratégie permet d'atteindre la configuration finale en 7 déplacements, ce qui correspond au nombre minimal théorique.

Ce travail met en évidence le rôle de la représentation des connaissances, des règles et de la stratégie de sélection dans le comportement d'un système expert.

**Mots-clés :** système expert, intelligence artificielle, base de connaissances, moteur d'inférence, chaînage avant, Tours de Hanoï, Python, NumPy.

# Table des matières

**Résumé** **i**

**Liste des figures** **v**

**Liste des tableaux** **vi**

**1 Introduction générale** **1**

1.1 Contexte général . . . . . 1

1.2 Présentation du problème . . . . . 1

1.2.1 Configuration initiale . . . . . 2

1.2.2 Configuration finale recherchée . . . . . 2

1.3 Problématique . . . . . 3

1.4 Objectifs du travail . . . . . 3

1.4.1 Objectif général . . . . . 3

1.4.2 Objectifs spécifiques . . . . . 3

1.5 Méthodologie adoptée . . . . . 4

1.6 Principe du système expert . . . . . 5

1.6.1 Les faits . . . . . 5

1.6.2 La base de connaissances . . . . . 5

1.6.3 Le moteur d'inférence . . . . . 5

1.6.4 La métarègle . . . . . 5

1.7 Périmètre et hypothèses . . . . . 6

**2 Représentation de l'état du jeu** **7**

2.1 Modélisation du problème . . . . . 7

2.2 La classe `Jeu_Hanoi` . . . . . 7

2.2.1 Définition . . . . . 7

2.2.2 Le tableau `pic` . . . 8
2.2.3 Le tableau `nombre_palet` . . . 8
2.3 Configuration initiale . . . 8
2.4 Configuration finale . . . 9
2.5 Définition des constantes du problème . . . 9
2.6 Choix de représentation . . . 9

**3 Base experte et fonctions de manipulation 11**
3.1 Vue d'ensemble . . . 11
3.2 Identification d'une situation . . . 11
3.2.1 Principe de l'encodage . . . 11
3.2.2 Implémentation . . . 12
3.2.3 Exemples . . . 12
3.3 Vérification de l'état d'un pic . . . 13
3.4 Vérification d'un déplacement . . . 13
3.4.1 Règles du jeu . . . 13
3.4.2 Implémentation . . . 13
3.5 Exécution d'un déplacement . . . 14
3.6 Détection des situations déjà vues . . . 15
3.7 Base de règles initiale . . . 15
3.8 Métarègle de priorité . . . 16

**4 Moteur d'inférence en chaînage avant 17**
4.1 Principe du chaînage avant . . . 17
4.2 Vérification de l'éligibilité d'une règle . . . 17
4.3 Boucle principale . . . 18
4.3.1 Algorithme . . . 18
4.3.2 Implémentation . . . 19
4.4 Compléments sur la stratégie . . . 20

**5 Expérimentation initiale 21**
5.1 Conditions de l'expérience . . . 21
5.2 Déroulement de la résolution . . . 21
5.3 Résultat obtenu . . . 22
5.4 Comparaison avec l'optimum théorique . . . 22

5.5 Analyse des limites . . . . . 23

**6 Optimisation de la stratégie 24**

6.1 Motivation . . . . . 24
6.2 Enrichissement de la base experte . . . . . 24
6.3 Nouvelle stratégie de sélection . . . . . 25
6.3.1 Cycle du plus petit palet . . . . . 25
6.3.2 Algorithme de sélection . . . . . 25
6.4 Implémentation . . . . . 25
6.5 Résultats obtenus . . . . . 26
6.6 Interprétation . . . . . 27

**7 Analyse et comparaison des résultats 28**

7.1 Synthèse des résultats . . . . . 28
7.2 Interprétation . . . . . 28
7.2.1 Sur la stratégie initiale . . . . . 28
7.2.2 Sur la stratégie optimisée . . . . . 28
7.3 Rôle de la stratégie d'inférence . . . . . 29
7.4 Limites de l'approche . . . . . 29

**8 Conclusion générale 30**

8.1 Bilan du travail . . . . . 30
8.2 Résultats obtenus . . . . . 30
8.3 Enseignements . . . . . 31
8.4 Perspectives . . . . . 31
8.5 Conclusion . . . . . 31

**A Code Python complet 32**

A.1 Imports et classe `Jeu_Hanoi` . . . . . 32
A.2 Fonctions de manipulation . . . . . 32
A.3 Base experte et moteur d'inférence . . . . . 34
A.4 Stratégie optimisée . . . . . 35

**Dépôt Git du projet 37**

# Table des figures

4.1 Boucle principale du moteur d'inférence . . . . . 17

# Liste des tableaux

1.1 Configuration initiale du problème . . . . . 2
1.2 Configuration finale recherchée . . . . . 2

3.1 Règles de la base experte initiale . . . . . 15

5.1 Déroulement de la résolution initiale (26 coups) . . . . . 22
5.2 Bilan de la résolution initiale . . . . . 23

6.1 Règles ajoutées à la base experte . . . . . 24
6.2 Résolution optimisée (7 coups) . . . . . 27

7.1 Comparaison des stratégies . . . . . 28

Chapitre 1

# Introduction générale

## 1.1 Contexte général

L'intelligence artificielle regroupe un ensemble de méthodes et de techniques permettant à des systèmes informatiques de réaliser des tâches nécessitant habituellement certaines capacités de raisonnement, de décision ou de résolution de problèmes.

Parmi les approches historiques de l'intelligence artificielle figurent les systèmes experts. Ces systèmes reposent sur la représentation explicite de connaissances sous forme de faits et de règles, ainsi que sur un mécanisme d'inférence permettant de déduire de nouvelles informations ou de sélectionner des actions à partir d'un état donné.

Contrairement à une approche fondée exclusivement sur l'apprentissage à partir de données, un système expert utilise une base de connaissances construite à partir de règles préalablement définies. Son comportement dépend ainsi des connaissances représentées et de la stratégie utilisée pour appliquer les règles.

Dans ce travail pratique, ces principes sont étudiés à travers le problème classique des Tours de Hanoï. Ce problème constitue un cadre simple et contrôlé pour observer le fonctionnement d'un système à base de règles, la gestion des états successifs et le rôle du moteur d'inférence dans la résolution.

## 1.2 Présentation du problème

Les Tours de Hanoï sont un problème de déplacement de disques entre plusieurs supports verticaux, appelés *pics*. Le problème étudié dans ce TP comprend trois pics et trois palets de tailles différentes.

Initialement, tous les palets sont empilés sur le pic de départ, dans l'ordre décroissant de leur taille. L'objectif est de transférer l'ensemble des palets vers le pic d'arrivée, en utilisant le troisième pic comme support intermédiaire si nécessaire.

La résolution doit respecter deux contraintes fondamentales :

— Un seul palet peut être déplacé à chaque étape.

— Un palet de grande taille ne peut jamais être placé sur un palet de taille plus petite.

Un déplacement est donc autorisé si le pic de départ n'est pas vide et si le pic d'arrivée est vide ou contient un palet de taille supérieure au palet déplacé.

### 1.2.1 Configuration initiale

Le problème utilise trois pics numérotés de 0 à 2 :

— Pic 0 : pic de départ.

— Pic 1 : pic intermédiaire.

— Pic 2 : pic d'arrivée.

Les trois palets sont numérotés selon leur taille : le palet 1 est le plus petit, le palet 2 est de taille intermédiaire et le palet 3 est le plus grand.

La situation initiale est représentée comme suit :

Table 1.1 – Configuration initiale du problème

| **Pic** | **Palets (du bas vers le haut)** |
|---|---|
| Pic 0 | [3, 2, 1] |
| Pic 1 | [0, 0, 0] |
| Pic 2 | [0, 0, 0] |

### 1.2.2 Configuration finale recherchée

La résolution est terminée lorsque les trois palets sont transférés sur le pic 2, en conservant leur ordre de taille :

Table 1.2 – Configuration finale recherchée

| **Pic** | **Palets (du bas vers le haut)** |
|---|---|
| Pic 0 | [0, 0, 0] |
| Pic 1 | [0, 0, 0] |
| Pic 2 | [3, 2, 1] |

Pour trois palets, le nombre minimal de déplacements nécessaires est de sept. Cette valeur constitue une référence théorique pour évaluer les résultats du système développé.

## 1.3 Problématique

La problématique de ce travail consiste à concevoir un système expert capable de résoudre automatiquement les Tours de Hanoï à partir d'une base de règles et d'un moteur d'inférence.

Il ne s'agit pas uniquement de trouver une succession de déplacements légaux, mais également de comprendre l'influence de la stratégie de sélection des règles sur le chemin suivi par le système.

La question principale peut être formulée ainsi :

> Comment concevoir un système expert fondé sur une base de règles et un moteur d'inférence en chaînage avant permettant de résoudre les Tours de Hanoï, tout en améliorant la stratégie de sélection des règles afin de réduire le nombre de déplacements ?

Cette problématique conduit à étudier plusieurs aspects : la représentation de l'état du jeu, la formalisation des règles, la détection des situations déjà rencontrées, la sélection d'une règle applicable et l'évaluation du nombre de déplacements effectués.

## 1.4 Objectifs du travail

### 1.4.1 Objectif général

L'objectif général de ce TP est de développer en Python un système expert capable de résoudre le problème des Tours de Hanoï en s'appuyant sur une représentation des connaissances, une base de règles et un moteur d'inférence en chaînage avant.

### 1.4.2 Objectifs spécifiques

Les objectifs spécifiques sont les suivants :

— Comprendre les principes de fonctionnement d'un système expert et de son moteur d'inférence.

— Représenter informatiquement les pics et les palets à l'aide de tableaux NumPy.

— Concevoir une méthode permettant d'identifier les différentes situations du jeu.

— Définir les règles qui autorisent les déplacements entre les pics.

— Développer les fonctions nécessaires à la vérification et à l'exécution des déplacements.

— Mémoriser les situations déjà explorées afin d'éviter les répétitions.

— Construire un moteur d'inférence utilisant le chaînage avant et une métarègle de priorité.

— Évaluer le comportement de la stratégie initiale et analyser ses limites.

— Proposer une amélioration de la base experte et de la méthode de sélection des règles.

— Comparer les résultats obtenus avec le nombre minimal théorique de déplacements.

## 1.5 Méthodologie adoptée

La démarche suivie dans ce travail est progressive. Elle consiste à construire et valider les composants du système expert avant de les intégrer au moteur d'inférence.

1. **Modélisation du problème :** définir les pics, les palets, les configurations initiale et finale ainsi que les contraintes de déplacement.
2. **Représentation des états :** construire une structure de données permettant de représenter la position des palets et le nombre de palets présents sur chaque pic.
3. **Développement des fonctions :** implémenter les fonctions de calcul de situation, de vérification des déplacements, d'exécution des mouvements et de détection des situations déjà étudiées.
4. **Construction de la base experte :** formaliser les règles de déplacement fournies dans le sujet et définir leur ordre de priorité.
5. **Développement du moteur d'inférence :** mettre en œuvre un mécanisme de chaînage avant qui teste les règles, sélectionne une règle éligible et actualise la situation du jeu.
6. **Expérimentation initiale :** exécuter le système avec la base experte initiale, compter les déplacements et vérifier l'atteinte de la situation finale.
7. **Optimisation :** enrichir la base experte avec deux règles supplémentaires, puis adapter la stratégie de sélection des règles.
8. **Évaluation :** comparer le nombre de déplacements des deux approches avec la valeur optimale théorique pour trois palets.

## 1.6 Principe du système expert

Le système développé s'appuie sur trois composants principaux : les faits, la base de connaissances et le moteur d'inférence.

### 1.6.1 Les faits

Les faits correspondent aux informations décrivant la situation courante du jeu. Ils indiquent notamment la position des palets sur les différents pics.

À chaque déplacement, ces informations sont mises à jour pour représenter le nouvel état du problème.

### 1.6.2 La base de connaissances

La base de connaissances contient les règles de déplacement qui définissent les actions que le système peut envisager.

Chaque règle est représentée par un pic de départ et un pic d'arrivée. Une règle ne peut être appliquée que si le déplacement correspondant respecte les contraintes des Tours de Hanoï.

### 1.6.3 Le moteur d'inférence

Le moteur d'inférence examine les règles de la base experte à partir de la situation courante. Il vérifie les conditions d'application, sélectionne une règle éligible et effectue le déplacement correspondant.

Le mécanisme utilisé est le **chaînage avant** : le système part de la situation initiale, applique successivement des règles et produit de nouveaux états jusqu'à atteindre la situation finale ou lorsqu'aucune règle ne peut être appliquée.

### 1.6.4 La métarègle

Lorsque plusieurs règles sont éligibles, une métarègle détermine celle qui doit être sélectionnée.

Dans la première version du système, la métarègle consiste à choisir la règle ayant le plus petit indice. Les règles sont donc examinées dans un ordre fixe et la première règle éligible est appliquée.

Dans la version améliorée, la méthode de sélection est adaptée afin de mieux guider les déplacements. Cette modification permet d'étudier l'impact de la stratégie d'inférence

sur le nombre total de coups nécessaires à la résolution.

## 1.7 Périmètre et hypothèses

Le travail présenté dans ce rapport porte sur l'implémentation d'un système expert pour une configuration précise des Tours de Hanoï.

Les hypothèses retenues pour l'expérimentation initiale sont les suivantes :

— Le problème comporte trois pics.

— Le nombre de palets est fixé à trois.

— Les palets sont de tailles distinctes : 1, 2 et 3.

— La configuration initiale place tous les palets sur le pic 0.

— La configuration cible place tous les palets sur le pic 2.

— Chaque action correspond au déplacement d'un seul palet.

— Les déplacements doivent respecter les contraintes du problème.

— Les situations déjà rencontrées sont mémorisées pour éviter les cycles.

L'optimisation étudiée repose sur une stratégie de sélection conçue pour le cas de trois palets. Le fait d'obtenir une solution optimale dans cette configuration ne constitue pas une garantie d'optimalité pour un nombre quelconque de palets ou de pics.

Chapitre 2

# Représentation de l'état du jeu

## 2.1 Modélisation du problème

Le problème des Tours de Hanoï est modélisé à l'aide d'une classe Python nommée `Jeu_Hanoi`. Cette classe regroupe l'ensemble des informations nécessaires pour décrire une configuration du jeu à un instant donné.

Deux structures de données sont utilisées :

— `pic` : une matrice NumPy de dimensions $3 \times 3$ représentant les palets présents sur chaque pic ;

— `nombre_palet` : un tableau NumPy de taille 3 indiquant le nombre de palets présents sur chaque pic.

## 2.2 La classe `Jeu_Hanoi`

### 2.2.1 Définition

La classe `Jeu_Hanoi` est définie comme suit :

```
1 class Jeu_Hanoi:
2     def __init__(self):
3         self.pic = np.zeros([3, 3], dtype=int)
4         self.nombre_palet = np.zeros(3, dtype=int)
5         pass
```

Listing 2.1 – Classe `Jeu_Hanoi`

### 2.2.2 Le tableau `pic`

Le tableau `pic` est une matrice NumPy de dimensions $3 \times 3$. Il représente les trois pics sur lesquels les palets peuvent être empilés.

— Le **premier indice** correspond au numéro du pic (0, 1 ou 2).

— Le **second indice** correspond à la position du palet sur le pic. La position 0 correspond à la base.

— La **valeur** de chaque case correspond soit à la taille d'un palet (1, 2 ou 3), soit à zéro si l'emplacement est vide.

Par exemple, `pic[b, 0]` contient le palet le plus bas du pic numéro `b`.

### 2.2.3 Le tableau `nombre_palet`

Le tableau `nombre_palet` est un tableau NumPy de taille 3. Chaque case indique combien de palets sont empilés sur le pic correspondant.

Par exemple, `nombre_palet[0]` donne le nombre de palets présents sur le pic 0.

## 2.3 Configuration initiale

À l'initialisation, les trois palets sont empilés sur le pic 0 dans l'ordre décroissant de leur taille. La situation est créée de la manière suivante :

```
jeu = Jeu_Hanoi()

jeu.nombre_palet[0] = 3

jeu.pic[0, 0] = 3
jeu.pic[0, 1] = 2
jeu.pic[0, 2] = 1
```

Listing 2.2 – Initialisation de la situation de départ

La matrice résultante est :

```
[[3 2 1]
 [0 0 0]
 [0 0 0]]
```

Le tableau `nombre_palet` vaut alors `[3, 0, 0]`.

## 2.4 Configuration finale

La configuration finale recherchée place tous les palets sur le pic 2, en conservant leur ordre décroissant :

```
Jeu_final = Jeu_Hanoi()

Jeu_final.nombre_palet[2] = 3

Jeu_final.pic[2, 0] = 3
Jeu_final.pic[2, 1] = 2
Jeu_final.pic[2, 2] = 1
```

Listing 2.3 – Définition de la situation finale

La matrice correspondante est :

```
[[0 0 0]
 [0 0 0]
 [3 2 1]]
```

## 2.5 Définition des constantes du problème

Afin de faciliter la lecture et la maintenance du code, il est possible de définir quelques constantes au début du programme :

```
NOMBRE_PICS  = 3
NOMBRE_PALETS = 3
PIC_DEPART   = 0
PIC_ARRIVEE  = 2
```

Listing 2.4 – Constantes du problème

Ces constantes seront utilisées dans la suite du rapport pour désigner respectivement le nombre de pics, le nombre de palets, le pic de départ et le pic d'arrivée.

## 2.6 Choix de représentation

La représentation choisie présente plusieurs avantages :

— Elle est compacte et facile à manipuler grâce à NumPy.

— Elle permet de comparer rapidement deux configurations (via une transformation en nombre unique, décrite au chapitre suivant).

— Elle se prête bien à la simulation d'un déplacement puisqu'il suffit de modifier les valeurs de deux cases et de deux compteurs.

Le choix d'un tableau à deux dimensions pour `pic` et d'un vecteur pour `nombre_palet` correspond exactement à la modélisation proposée dans l'énoncé du TP.

Chapitre 3

# Base experte et fonctions de manipulation

## 3.1 Vue d'ensemble

La base experte regroupe l'ensemble des fonctions permettant de manipuler une configuration du jeu : identification d'une situation, vérification des conditions de déplacement, exécution d'un mouvement et détection des situations déjà étudiées.

Ces fonctions constituent les briques élémentaires utilisées ensuite par le moteur d'inférence.

## 3.2 Identification d'une situation

### 3.2.1 Principe de l'encodage

Pour éviter de boucler indéfiniment, le système doit être capable de reconnaître si une configuration a déjà été rencontrée. Chaque situation est donc associée à un nombre entier unique.

L'encodage utilisé repose sur les puissances de 2 :

— le palet de taille 1 correspond à $2^0$ ;

— le palet de taille 2 correspond à $2^1$ ;

— le palet de taille 3 correspond à $2^2$.

Un décalage de $3 \times b$ est ensuite appliqué en fonction du numéro du pic $b$. Chaque palet de taille $t$ situé sur le pic $b$ contribue donc à la valeur :

$$2^{(t-1)+3b}$$

Le nombre total est la somme de ces contributions pour l'ensemble des palets présents.

### 3.2.2 Implémentation

```python
def nombre_situation(jeu):
    nombre = 0

    # Premi re boucle : parcours des trois pics
    for b in range(3):

        # Deuxi me boucle : parcours des palets du pic b
        for palet in range(jeu.nombre_palet[b]):

            # Taille du palet courant
            taille = jeu.pic[b, palet]

            # Contribution du palet au nombre global
            nombre += 2 ** (taille - 1 + b * 3)

    return nombre
```

Listing 3.1 – Fonction `nombre_situation`

### 3.2.3 Exemples

À l'initialisation, la situation est :

```
[[3 2 1]
 [0 0 0]
 [0 0 0]]
```

Le nombre associé vaut $2^0 + 2^1 + 2^2 = 7$, soit en binaire `000000111`.

À la situation finale :

```
[[0 0 0]
 [0 0 0]
 [3 2 1]]
```

Le nombre associé vaut $2^6 + 2^7 + 2^8 = 448$, soit en binaire `111000000`.

## 3.3 Vérification de l'état d'un pic

La fonction `pic_vide` vérifie si un pic ne contient aucun palet :

```
def pic_vide(indice_pic, jeu):
    reponse = jeu.nombre_palet[indice_pic] == 0
    return reponse
```

Listing 3.2 – Fonction `pic_vide`

Elle retourne `True` si le pic est vide et `False` sinon.

## 3.4 Vérification d'un déplacement

### 3.4.1 Règles du jeu

Un déplacement du pic `pic1` vers le pic `pic2` est autorisé si :

1. le pic de départ contient au moins un palet ;
2. le pic d'arrivée est vide, ou son palet supérieur est plus grand que le palet déplacé.

### 3.4.2 Implémentation

```
def regle_jeu(indice_pic1, indice_pic2, jeu):

    # Le pic de d part doit contenir au moins un palet
    if jeu.nombre_palet[indice_pic1] == 0:
        return False

    # Si le pic d'arriv e est vide, le d placement est l gal
    if jeu.nombre_palet[indice_pic2] == 0:
        return True

    # Palet sup rieur du pic de d part
    palet_depart = jeu.pic[
        indice_pic1,
        jeu.nombre_palet[indice_pic1] - 1
    ]

    # Palet sup rieur du pic d'arriv e
    palet_arrivee = jeu.pic[
```

```
        indice_pic2,
        jeu.nombre_palet[indice_pic2] - 1
    ]

    # Le palet d plac  doit  tre  plus petit
    if palet_depart < palet_arrivee:
        return True

    return False
```

Listing 3.3 – Fonction `regle_jeu`

## 3.5 Exécution d'un déplacement

La fonction `effectue_deplacement` réalise concrètement le déplacement d'un palet. Elle met à jour la matrice `pic` et le tableau `nombre_palet`.

```
def effectue_deplacement(indice_pic1, indice_pic2, jeu):

    # Position du palet    d placer
    position_depart = jeu.nombre_palet[indice_pic1] - 1

    # R cup ration du palet
    palet = jeu.pic[indice_pic1, position_depart]

    # Retrait du palet du pic de d part
    jeu.pic[indice_pic1, position_depart] = 0
    jeu.nombre_palet[indice_pic1] -= 1

    # Position d'arriv e du palet
    position_arrivee = jeu.nombre_palet[indice_pic2]

    # Placement du palet sur le pic d'arriv e
    jeu.pic[indice_pic2, position_arrivee] = palet
    jeu.nombre_palet[indice_pic2] += 1
```

Listing 3.4 – Fonction `effectue_deplacement`

## 3.6 Détection des situations déjà vues

Afin d'éviter les boucles, le système vérifie que le déplacement envisagé ne conduit pas à une situation déjà étudiée. Pour cela, le déplacement est simulé sur une copie du jeu.

```
def situation_vue(indice_pic1, indice_pic2,
                  jeu, situation_etudiee):

    # Vrification que le dplacement est l gal
    if not regle_jeu(indice_pic1, indice_pic2, jeu):
        return False

    # Copie ind pendante du jeu
    jeu_tmp = Jeu_Hanoi()
    jeu_tmp.pic = copy.deepcopy(jeu.pic)
    jeu_tmp.nombre_palet = copy.deepcopy(jeu.nombre_palet)

    # Simulation du dplacement sur la copie
    effectue_deplacement(indice_pic1, indice_pic2, jeu_tmp)

    # Identifiant de la nouvelle situation
    nombre_nouvelle_situation = nombre_situation(jeu_tmp)

    # True si la situation est nouvelle
    reponse = nombre_nouvelle_situation not in situation_etudiee

    return reponse
```

Listing 3.5 – Fonction `situation_vue`

## 3.7 Base de règles initiale

La base experte initiale comporte quatre règles de déplacement, fournies par l'expert :

TABLE 3.1 – Règles de la base experte initiale

| Règle | Départ | Arrivée |
|---|---|---|
| R1 | Pic 0 | Pic 1 |
| R2 | Pic 1 | Pic 2 |
| R3 | Pic 2 | Pic 1 |
| R4 | Pic 1 | Pic 0 |

```
regles = [
    (0, 1),   # R1 : pic 0 -> pic 1
    (1, 2),   # R2 : pic 1 -> pic 2
    (2, 1),   # R3 : pic 2 -> pic 1
    (1, 0)    # R4 : pic 1 -> pic 0
]
```

Listing 3.6 – Base experte initiale

## 3.8 Métarègle de priorité

Lorsque plusieurs règles sont éligibles simultanément, une métarègle détermine celle qui sera appliquée. La métarègle initiale impose de choisir la règle ayant le plus petit indice :

> Parmi les règles éligibles, sélectionner celle dont l'indice est le plus petit. Les règles sont donc testées dans l'ordre R1, R2, R3, R4 et la première règle éligible est appliquée.

Cette métarègle présente l'avantage d'être simple à implémenter : il suffit de parcourir la liste `regles` dans son ordre naturel et de s'arrêter dès qu'une règle est éligible.

Chapitre 4

# Moteur d'inférence en chaînage avant

## 4.1 Principe du chaînage avant

Le moteur d'inférence utilise le mécanisme de **chaînage avant**. Il part de la situation initiale du jeu, sélectionne une règle éligible dans la base experte, applique le déplacement correspondant puis recommence.

Le processus s'arrête lorsque la situation finale est atteinte, ou bien lorsqu'aucune règle ne peut être appliquée.

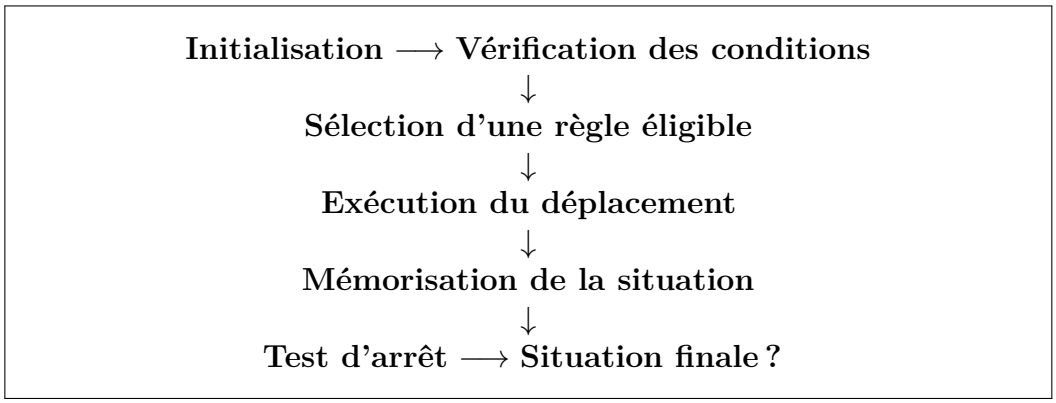



FIGURE 4.1 – Boucle principale du moteur d'inférence

## 4.2 Vérification de l'éligibilité d'une règle

Une règle est éligible si les trois conditions suivantes sont réunies :

1. le pic de départ contient au moins un palet ;
2. le déplacement est autorisé par les règles du jeu ;
3. la situation résultante n'a pas déjà été étudiée.

```python
def regle_eligible(indice_pic1, indice_pic2,
                   jeu, situation_etudiee):

    # Le pic de d part doit contenir un palet
    if jeu.nombre_palet[indice_pic1] == 0:
        return False

    # Le d placement doit  tre  l gal
    if not regle_jeu(indice_pic1, indice_pic2, jeu):
        return False

    # La nouvelle situation doit  tre  in dite
    if not situation_vue(
        indice_pic1, indice_pic2,
        jeu, situation_etudiee
    ):
        return False

    return True
```

Listing 4.1 – Fonction `regle_eligible`

## 4.3 Boucle principale

### 4.3.1 Algorithme

L'algorithme du moteur d'inférence est le suivant :

1. Calculer l'identifiant de la situation initiale et l'ajouter à la liste des situations étudiées.
2. Calculer l'identifiant de la situation finale.
3. Initialiser le compteur de coups à zéro.
4. Tant que la situation courante n'est pas la situation finale :
   (a) Parcourir les règles dans l'ordre de priorité.
   (b) Si la règle est éligible, appliquer le déplacement, mémoriser la nouvelle situation, incrémenter le compteur et interrompre la boucle des règles.
   (c) Si aucune règle n'est éligible, arrêter le moteur.

### 4.3.2 Implémentation

```
# Initialisation du jeu
jeu = Jeu_Hanoi()
jeu.nombre_palet[0] = 3
jeu.pic[0, 0] = 3
jeu.pic[0, 1] = 2
jeu.pic[0, 2] = 1

# Situation initiale
situation_initiale = nombre_situation(jeu)
situation_etudiee = [situation_initiale]

# Situation finale
jeu_final = Jeu_Hanoi()
jeu_final.nombre_palet[2] = 3
jeu_final.pic[2, 0] = 3
jeu_final.pic[2, 1] = 2
jeu_final.pic[2, 2] = 1
situation_finale = nombre_situation(jeu_final)

# Compteur
nombre_coup = 0

# Boucle principale
while nombre_situation(jeu) != situation_finale:

    regle_appliquee = False

    for indice_regle, regle in enumerate(regles, start=1):

        indice_pic1 = regle[0]
        indice_pic2 = regle[1]

        if regle_eligible(
            indice_pic1, indice_pic2,
            jeu, situation_etudiee
        ):

            effectue_deplacement(
                indice_pic1, indice_pic2, jeu
```

```
            )

            nouvelle_situation = nombre_situation(jeu)
            situation_etudiee.append(nouvelle_situation)

            nombre_coup += 1

            print(f"Coup {nombre_coup}")
            print(f"R gle {indice_regle} : "
                  f"pic {indice_pic1} -> pic {indice_pic2}")
            print(jeu.pic)
            print()

            regle_appliquee = True
            break

    if not regle_appliquee:
        print("Aucune r gle applicable.")
        break
```

Listing 4.2 – Moteur d'inférence en chaînage avant

## 4.4 Compléments sur la stratégie

La métarègle choisit systématiquement la première règle éligible dans l'ordre R1, R2, R3, R4. Cette stratégie permet de trouver une solution valide, mais elle n'intègre aucune information sur la progression vers la situation finale.

Il en résulte un comportement purement local : le moteur applique la première règle possible sans évaluer si ce choix rapproche ou non le système de l'objectif. Le prochain chapitre présente les résultats obtenus avec cette stratégie et analyse ses limites.

Chapitre 5

# Expérimentation initiale

## 5.1 Conditions de l'expérience

L'expérimentation est réalisée avec la configuration suivante :

— trois pics (0, 1 et 2) ;

— trois palets de tailles 1, 2 et 3 ;

— configuration initiale : tous les palets sur le pic 0 ;

— configuration finale : tous les palets sur le pic 2 ;

— base experte initiale : règles R1 à R4 ;

— métarègle : choix de la règle de plus petit indice.

## 5.2 Déroulement de la résolution

Les vingt-six coups produits par le moteur sont présentés dans le tableau 5.1.

TABLE 5.1 – Déroulement de la résolution initiale (26 coups)

| Coup | Règle | Déplacement |
|---|---|---|
| 1 | R1 | $0 \to 1$ |
| 2 | R2 | $1 \to 2$ |
| 3 | R1 | $0 \to 1$ |
| 4 | R3 | $2 \to 1$ |
| 5 | R4 | $1 \to 0$ |
| 6 | R2 | $1 \to 2$ |
| 7 | R1 | $0 \to 1$ |
| 8 | R2 | $1 \to 2$ |
| 9 | R1 | $0 \to 1$ |
| 10 | R3 | $2 \to 1$ |
| 11 | R4 | $1 \to 0$ |
| 12 | R3 | $2 \to 1$ |
| 13 | R1 | $0 \to 1$ |
| 14 | R2 | $1 \to 2$ |
| 15 | R4 | $1 \to 0$ |
| 16 | R3 | $2 \to 1$ |
| 17 | R4 | $1 \to 0$ |
| 18 | R2 | $1 \to 2$ |
| 19 | R1 | $0 \to 1$ |
| 20 | R2 | $1 \to 2$ |
| 21 | R1 | $0 \to 1$ |
| 22 | R3 | $2 \to 1$ |
| 23 | R4 | $1 \to 0$ |
| 24 | R2 | $1 \to 2$ |
| 25 | R1 | $0 \to 1$ |
| 26 | R2 | $1 \to 2$ |

## 5.3 Résultat obtenu

À l'issue de ces 26 déplacements, la situation finale est atteinte :

```
[[0 0 0]
 [0 0 0]
 [3 2 1]]
```

Le nombre de palets sur chaque pic vaut `[0, 0, 3]`.

## 5.4 Comparaison avec l'optimum théorique

Pour trois palets, le nombre minimal de déplacements nécessaires est donné par la formule :

$$N_{\min} = 2^n - 1 = 2^3 - 1 = 7$$

Le tableau 5.2 présente le bilan de la résolution initiale.

TABLE 5.2 – Bilan de la résolution initiale

| **Indicateur** | **Valeur** |
|---|---|
| Nombre de coups effectués | 26 |
| Nombre optimal | 7 |
| Écart à l'optimum | 19 |
| Facteur par rapport à l'optimum | $\approx 3{,}71$ |

## 5.5 Analyse des limites

Le système expert atteint donc bien la configuration finale, mais avec un nombre de déplacements presque quatre fois supérieur à l'optimum. Cette différence s'explique principalement par la métarègle utilisée.

La métarègle initiale sélectionne la première règle éligible sans tenir compte de la progression vers l'objectif. Le moteur applique donc des règles localement valides, mais dont l'enchaînement conduit à effectuer des déplacements superflus.

De plus, la base experte initiale ne contient pas les déplacements directs entre les pics 0 et 2. Cette limitation prive le moteur de certains raccourcis et favorise l'apparition de cycles inutiles.

Chapitre 6

# Optimisation de la stratégie

## 6.1 Motivation

La résolution initiale atteint la situation finale en 26 coups, alors que l'optimum théorique est de 7 coups. Une amélioration est donc nécessaire.

Deux pistes sont envisagées :

1. enrichir la base experte avec les déplacements directs entre les pics 0 et 2 ;
2. adapter la stratégie de sélection des règles en s'appuyant sur la position du plus petit palet.

## 6.2 Enrichissement de la base experte

On ajoute deux règles supplémentaires :

Table 6.1 – Règles ajoutées à la base experte

| Règle | Départ | Arrivée |
|---|---|---|
| R5 | Pic 0 | Pic 2 |
| R6 | Pic 2 | Pic 0 |

```
regles_ameliorees = [
    (0, 1),   # R1
    (1, 2),   # R2
    (2, 1),   # R3
    (1, 0),   # R4
    (0, 2),   # R5
    (2, 0)    # R6
]
```

Listing 6.1 – Base experte améliorée

## 6.3 Nouvelle stratégie de sélection

L'ajout de règles ne suffit pas à lui seul à obtenir une solution optimale : la stratégie de sélection doit également être adaptée.

La nouvelle stratégie repose sur l'observation classique suivante : dans la solution optimale des Tours de Hanoï, le plus petit palet se déplace tous les deux coups en suivant un cycle entre les trois pics.

### 6.3.1 Cycle du plus petit palet

Pour un transfert du pic 0 vers le pic 2, le cycle optimal du plus petit palet est :

$$0 \rightarrow 2 \rightarrow 1 \rightarrow 0$$

### 6.3.2 Algorithme de sélection

1. Localiser le pic contenant le plus petit palet.
2. Si le numéro du coup est impair, déplacer obligatoirement le plus petit palet vers le pic suivant dans le cycle.
3. Si le numéro du coup est pair, déplacer un autre palet (le plus grand possible parmi les déplacements légaux).

## 6.4 Implémentation

```
def choisir_regle_amelioree(jeu, numero_coup):

    # Localisation du plus petit palet
    position_petit = None
    for pic in range(3):
        for position in range(3):
            if jeu.pic[pic, position] == 1:
                position_petit = pic

    # Cycle du plus petit palet
```

```
    destination = {
        0: 2,
        2: 1,
        1: 0
    }

    # Coups impairs : d placer le plus petit palet
    if numero_coup % 2 == 1:
        pic_depart = position_petit
        pic_arrivee = destination[position_petit]

        for indice, (depart, arrivee) in enumerate(
            regles_ameliorees, start=1
        ):
            if depart == pic_depart and arrivee == pic_arrivee:
                if regle_jeu(depart, arrivee, jeu):
                    return indice

    # Coups pairs : d placer un autre palet
    for indice, (depart, arrivee) in enumerate(
        regles_ameliorees, start=1
    ):
        if depart != position_petit:
            if regle_jeu(depart, arrivee, jeu):
                return indice

    return None
```

Listing 6.2 – Fonction `choisir_regle_amelioree`

## 6.5 Résultats obtenus

La nouvelle stratégie permet d'atteindre la situation finale en 7 coups. La séquence obtenue est présentée dans le tableau 6.2.

Table 6.2 – Résolution optimisée (7 coups)

| Coup | Règle | Déplacement |
|---|---|---|
| 1 | R5 | $0 \rightarrow 2$ |
| 2 | R1 | $0 \rightarrow 1$ |
| 3 | R3 | $2 \rightarrow 1$ |
| 4 | R5 | $0 \rightarrow 2$ |
| 5 | R4 | $1 \rightarrow 0$ |
| 6 | R2 | $1 \rightarrow 2$ |
| 7 | R5 | $0 \rightarrow 2$ |

À l'issue de ces sept déplacements, la configuration finale est obtenue :

```
[[0 0 0]
 [0 0 0]
 [3 2 1]]
```

## 6.6 Interprétation

La séquence obtenue correspond exactement à l'optimum théorique pour trois palets. Le fait d'utiliser une règle de déplacement direct entre les pics 0 et 2 (règle R5) permet de respecter le cycle naturel du plus petit palet dans la solution optimale.

Ce résultat montre qu'une base de règles plus riche ne suffit pas à elle seule : c'est la stratégie de sélection qui détermine la qualité de la résolution.

Chapitre 7

# Analyse et comparaison des résultats

## 7.1 Synthèse des résultats

Le tableau 7.1 présente la comparaison entre les deux stratégies et l'optimum théorique.

Table 7.1 – Comparaison des stratégies

| **Stratégie** | **Coups** | **Facteur / optimal** |
|---|---|---|
| Stratégie initiale | 26 | $\approx 3{,}71$ |
| Stratégie optimisée | 7 | 1,00 |
| Optimum théorique | 7 | 1,00 |

## 7.2 Interprétation

### 7.2.1 Sur la stratégie initiale

La stratégie initiale explore les règles dans l'ordre R1, R2, R3, R4 et applique la première règle éligible. Elle produit une solution correcte mais coûteuse en coups.

L'absence de règles directes entre les pics 0 et 2 contraint le moteur à utiliser le pic 1 comme intermédiaire obligatoire pour chaque échange entre les pics extrêmes. Cela allonge artificiellement la solution.

### 7.2.2 Sur la stratégie optimisée

La stratégie optimisée combine deux modifications :

— l'ajout des règles R5 et R6 qui autorisent les déplacements directs entre les pics 0 et 2 ;

— la sélection des règles en fonction de la parité du coup et de la position du plus petit palet.

Cette approche reproduit la structure de la solution optimale des Tours de Hanoï, ce qui conduit à un nombre de coups égal à l'optimum théorique.

## 7.3 Rôle de la stratégie d'inférence

L'expérience met en évidence le rôle central de la stratégie d'inférence dans un système expert :

— les règles définissent les actions possibles ;

— la métarègle détermine la qualité de l'exploration ;

— une même base de règles, associée à deux stratégies différentes, peut produire des solutions très différentes en termes de performance.

## 7.4 Limites de l'approche

L'optimisation proposée est spécifique au cas à trois palets et à trois pics. Le cycle $0 \to 2 \to 1 \to 0$ du plus petit palet dépend de la configuration initiale et de la configuration cible.

Pour généraliser cette démarche à d'autres tailles, il faudrait :

— adapter le cycle du plus petit palet ;

— enrichir la base experte en conséquence ;

— envisager une stratégie de recherche plus générale (par exemple une recherche en profondeur avec heuristique).

Chapitre 8

# Conclusion générale

## 8.1 Bilan du travail

Ce travail pratique a permis de concevoir et d'implémenter un système expert capable de résoudre le problème des Tours de Hanoï à l'aide d'un moteur d'inférence en chaînage avant.

Le système repose sur les composants suivants :

— une modélisation explicite des états du jeu à l'aide de tableaux NumPy ;

— une fonction d'encodage des situations en nombres uniques, permettant d'éviter les cycles ;

— des fonctions de vérification et d'exécution des déplacements ;

— une base experte de règles de déplacement ;

— une métarègle de sélection ;

— un moteur d'inférence en chaînage avant.

## 8.2 Résultats obtenus

Deux stratégies ont été mises en œuvre :

— une stratégie initiale, fondée sur un ordre de priorité fixe, qui atteint la situation finale en 26 coups ;

— une stratégie optimisée, combinant l'ajout de nouvelles règles et une sélection fondée sur le cycle du plus petit palet, qui atteint la situation finale en 7 coups.

La seconde stratégie atteint l'optimum théorique pour trois palets.

## 8.3 Enseignements

Ce travail met en évidence plusieurs points importants relatifs aux systèmes experts :

1. la représentation des connaissances joue un rôle essentiel dans la capacité du système à trouver une solution ;
2. la qualité de la base de règles ne suffit pas : la stratégie d'inférence détermine la performance effective ;
3. la mémorisation des situations déjà vues est indispensable pour éviter les cycles et garantir la terminaison.

## 8.4 Perspectives

Plusieurs extensions peuvent être envisagées :

— **Généralisation à plus de palets :** étendre le système pour traiter quatre ou cinq palets, en vérifiant que la stratégie reste optimale.

— **Ajout d'un quatrième pic :** étudier l'impact d'un pic supplémentaire sur la base experte et la stratégie.

— **Comparaison avec l'algorithme récursif classique :** mesurer l'écart entre la solution du système expert et la solution récursive optimale.

— **Approche heuristique :** utiliser une fonction d'évaluation pour guider la sélection des règles (par exemple le nombre de palets bien placés).

— **Apprentissage de règles :** envisager une approche hybride où certaines règles seraient apprises à partir d'expériences.

## 8.5 Conclusion

Le système expert développé respecte l'ensemble des contraintes du problème et atteint automatiquement la configuration finale. La comparaison entre les deux stratégies met en évidence l'importance de la stratégie d'inférence : à base de règles équivalente, une organisation différente des règles produit des résultats radicalement différents.

Ce travail illustre ainsi, sur un cas simple et contrôlé, les principes fondamentaux des systèmes experts : représentation explicite des connaissances, raisonnement par règles et rôle central du moteur d'inférence.

Chapitre A

# Code Python complet

## A.1 Imports et classe Jeu_Hanoi

```
import numpy as np
import copy

class Jeu_Hanoi:
    def __init__(self):
        self.pic = np.zeros([3, 3], dtype=int)
        self.nombre_palet = np.zeros(3, dtype=int)
        pass
```

Listing A.1 – Imports et classe principale

## A.2 Fonctions de manipulation

```
def nombre_situation(jeu):
    nombre = 0
    for b in range(3):
        for palet in range(jeu.nombre_palet[b]):
            taille = jeu.pic[b, palet]
            nombre += 2 ** (taille - 1 + b * 3)
    return nombre

def pic_vide(indice_pic, jeu):
    return jeu.nombre_palet[indice_pic] == 0

def regle_jeu(indice_pic1, indice_pic2, jeu):
    if jeu.nombre_palet[indice_pic1] == 0:
```

```
14          return False
15      if jeu.nombre_palet[indice_pic2] == 0:
16          return True
17      palet_depart = jeu.pic[
18          indice_pic1, jeu.nombre_palet[indice_pic1] - 1
19      ]
20      palet_arrivee = jeu.pic[
21          indice_pic2, jeu.nombre_palet[indice_pic2] - 1
22      ]
23      return palet_depart < palet_arrivee
24
25  def effectue_deplacement(indice_pic1, indice_pic2, jeu):
26      position_depart = jeu.nombre_palet[indice_pic1] - 1
27      palet = jeu.pic[indice_pic1, position_depart]
28      jeu.pic[indice_pic1, position_depart] = 0
29      jeu.nombre_palet[indice_pic1] -= 1
30
31      position_arrivee = jeu.nombre_palet[indice_pic2]
32      jeu.pic[indice_pic2, position_arrivee] = palet
33      jeu.nombre_palet[indice_pic2] += 1
34
35  def situation_vue(indice_pic1, indice_pic2,
36                    jeu, situation_etudiee):
37      if not regle_jeu(indice_pic1, indice_pic2, jeu):
38          return False
39
40      jeu_tmp = Jeu_Hanoi()
41      jeu_tmp.pic = copy.deepcopy(jeu.pic)
42      jeu_tmp.nombre_palet = copy.deepcopy(jeu.nombre_palet)
43
44      effectue_deplacement(indice_pic1, indice_pic2, jeu_tmp)
45      nombre_nouvelle_situation = nombre_situation(jeu_tmp)
46      return nombre_nouvelle_situation not in situation_etudiee
47
48  def regle_eligible(indice_pic1, indice_pic2,
49                     jeu, situation_etudiee):
50      if jeu.nombre_palet[indice_pic1] == 0:
51          return False
52      if not regle_jeu(indice_pic1, indice_pic2, jeu):
53          return False
54      if not situation_vue(
```

```
        indice_pic1, indice_pic2,
        jeu, situation_etudiee
    ):
        return False
    return True
```

Listing A.2 – Fonctions utilitaires

## A.3 Base experte et moteur d'inférence

```
regles = [
    (0, 1),   # R1
    (1, 2),   # R2
    (2, 1),   # R3
    (1, 0)    # R4
]

jeu = Jeu_Hanoi()
jeu.nombre_palet[0] = 3
jeu.pic[0, 0] = 3
jeu.pic[0, 1] = 2
jeu.pic[0, 2] = 1

situation_initiale = nombre_situation(jeu)
situation_etudiee = [situation_initiale]

jeu_final = Jeu_Hanoi()
jeu_final.nombre_palet[2] = 3
jeu_final.pic[2, 0] = 3
jeu_final.pic[2, 1] = 2
jeu_final.pic[2, 2] = 1
situation_finale = nombre_situation(jeu_final)

nombre_coup = 0

while nombre_situation(jeu) != situation_finale:
    regle_appliquee = False
    for indice_regle, regle in enumerate(regles, start=1):
        indice_pic1, indice_pic2 = regle
        if regle_eligible(
            indice_pic1, indice_pic2,
```

```
            jeu, situation_etudiee
        ):
            effectue_deplacement(
                indice_pic1, indice_pic2, jeu
            )
            situation_etudiee.append(
                nombre_situation(jeu)
            )
            nombre_coup += 1
            regle_appliquee = True
            break
    if not regle_appliquee:
        break
```

Listing A.3 – Base experte et moteur (version initiale)

## A.4 Stratégie optimisée

```
regles_ameliorees = [
    (0, 1), (1, 2), (2, 1),
    (1, 0), (0, 2), (2, 0)
]

def choisir_regle_amelioree(jeu, numero_coup):
    position_petit = None
    for pic in range(3):
        for position in range(3):
            if jeu.pic[pic, position] == 1:
                position_petit = pic

    destination = {0: 2, 2: 1, 1: 0}

    if numero_coup % 2 == 1:
        pic_depart = position_petit
        pic_arrivee = destination[position_petit]
        for indice, (depart, arrivee) in enumerate(
            regles_ameliorees, start=1
        ):
            if depart == pic_depart and arrivee == pic_arrivee:
                if regle_jeu(depart, arrivee, jeu):
                    return indice
```

```
24
25      for indice, (depart, arrivee) in enumerate(
26          regles_ameliorees, start=1
27      ):
28          if depart != position_petit:
29              if regle_jeu(depart, arrivee, jeu):
30                  return indice
31      return None
```

Listing A.4 – Stratégie optimisée

# Dépôt Git du projet

L'ensemble du code source associé à ce rapport (notebooks, scripts Python et éventuels fichiers de données) est disponible sur le dépôt Git suivant :

`https://github.com/houda-hajjaj/Machine-Learning.git`

Le dépôt contient notamment :

— le notebook complet de résolution des Tours de Hanoï ;

— les différentes stratégies d'inférence étudiées ;

— la base experte et le moteur d'inférence en chaînage avant ;

— les scripts utilisés pour reproduire les résultats présentés dans ce rapport.